
GRAFIKA KOMPUTEROWA LAB_5

Table of Contents

1. CEL CWICZENIA	1
2. ZADANIA	1
2.0 Funkcje pomocnicze	1
2.0.1 Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie z poprzednich laboratoriów	1
2.0.2 Funkcja do tworzenia macierzy transformacji dla obrotu i przesunięcia w przestrzeni 3D	2
2.0.3 Funkcja do tworzenia macierzy transformacji dla rzutu perspektywicznego	2
2.0.4 Funkcja do rysowania linii na obrazie monochromatycznym	2
2.0.5 Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty $p1, p2, p3$	4
2.0.6 Funkcja do wyznaczania odległości punktu $[x, y, z]$ od płaszczyzny o równaniu $Ax+By+Cz+D=0$	4
2.0.7 Funkcja pomocnicza z laboratorium 04 wypełniająca kontur	7
2.0.8 Funkcja pomocnicza z laboratorium 4 rysująca trójkąt i wypełniająca go	13
2.1 ZADANIE 1	13
2.3 ZADANIE 3	24
Typowy koniec skryptu	25

Joanna Dagil 231008

1. CEL CWICZENIA

Segmentacja obrazów (algorytm watershed), eliminacja elementów zasłoniętych.

2. ZADANIA

2.0 Funkcje pomocnicze

```
clear; close all; clc;
```

2.0.1 Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie z poprzednich laboratoriów

```
function [i,j] = mat_to_pix(x, y, M, N, dx, dy)
    T = [1/dx, 0, M/2;
         0, -1/dy, N/2;
         0, 0, 1];
    temp = [x; y; 1];
    res = T * temp;
    i = round(res(1)/res(3));
    j = round(res(2)/res(3));
end
```

```
function [i,j] = pix_to_mat(x, y, M, N, dx, dy)
    T = [dx, 0, -M*dx/2;
         0, -dy, N*dy/2;
         0, 0, 1];
    res = T * [x; y; 1];
    i = res(1)/res(3);
    j = res(2)/res(3);
end
```

2.0.2 Funkcja do tworzenia macierzy transformacji dla obrotu i przesunięcia w przestrzeni 3D

```
function Hmat = RPYT(roll,pitch,yaw,px,py,pz)
r = roll*pi/180; p = pitch*pi/180; y = yaw*pi/180;
MR = [cos(r), -sin(r), 0;
      sin(r), cos(r), 0;
      0, 0, 1];
MP = [cos(p), 0, sin(p);
      0, 1, 0;
      -sin(p), 0, cos(p)];
MY = [1, 0, 0;
      0, cos(y), -sin(y);
      0, sin(y), cos(y)];
Rot = MY*MP*MR;
Hmat = zeros(4,4);
Hmat(1:3,1:3)=Rot;
Hmat(1,4)=px; Hmat(2,4)=py; Hmat(3,4)=pz;
Hmat(4,4)=1;
end
```

2.0.3 Funkcja do tworzenia macierzy transformacji dla rzutu perspektywicznego

```
function H = Persp(f)
% zwraca macierz jednorodna transformacji rzutu perspektywicznego
% z użyciem kamery o osi OZ i o ogniskowej f (konwencja jak na wykładzie)
H = zeros(4,4);
H(1,1)=1; H(2,2)=1;
H(4,3)=-1/f; H(4,4)=1;
end
```

2.0.4 Funkcja do rysowania linii na obrazie monochromatycznym

```
function Im2 = linia1(Im1,yp,xp,yk,xk,C)
% na monochromatycznym obrazie kresli linie o jasności C
```

```
% od punktu (yp, xp) do punktu (yk, xk)
[Y, X] = size(Im1);
Im2 = Im1;

if (abs(yk-yp) > abs(xk-xp))           % stroma linia
    x0 = yp; y0 = xp; x1 = yk; y1 = xk; % zamiana wspolrzecznych
    zamiana = 1;
else
    x0 = xp; y0 = yp; x1 = xk; y1 = yk;
    zamiana = 0;
end
if zamiana==1
    temp=X;
    X=Y;
    Y=temp;
end % zamiana rozmiarow X i Y
if(x0 > x1) % zamiana poczatku i konca linii, zeby zaczynac od lewej
    temp1 = x0; x0 = x1; x1 = temp1;
    temp2 = y0; y0 = y1; y1 = temp2;
end
dx = abs(x1 - x0) ;                    % odleglosc x
dy = abs(y1 - y0);                    % odleglosc y
sy = sign(y1 - y0);                   % znak przyrostu w kierunku y
x = x0; y = y0;                       % inicjalizacja
if x>0 && x<=X && y>0 && y<=Y
if (zamiana ==0)
    Im2(y, x) = C;                    % rysowanie punktu
else
    Im2(x, y) = C;
end
end
error = 2*dy - dx ;                   % inicjalizacja bledu
for i = 0:dx-1
    error = error + 2*dy;              % modyfikacja bledu
    if (error >0)
        y = y +sy;
        error = error - 2*dx;
    end
    x = x + 1;                         % zwiekszamy x
    if x>0 && x<=X && y>0 && y<=Y
        if (zamiana ==0)
            Im2(y, x) = C;            % rysowanie punktu
        else
            Im2(x, y) = C;
        end
    end
end
end
end
```

2.0.5 Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty $p1, p2, p3$

```
function [A, B, C, D] = plane(p1,p2,p3)
    A = det([p1(2),p1(3),1;
             p2(2),p2(3),1;
             p3(2),p3(3),1]);
    B = -det([p1(1),p1(3),1;
              p2(1),p2(3),1;
              p3(1),p3(3),1]);
    C = det([p1(1),p1(2),1;
             p2(1),p2(2),1;
             p3(1),p3(2),1]);
    D = -det([p1(1),p1(2),p1(3);
              p2(1),p2(2),p2(3);
              p3(1),p3(2),p3(3)]);
end
```

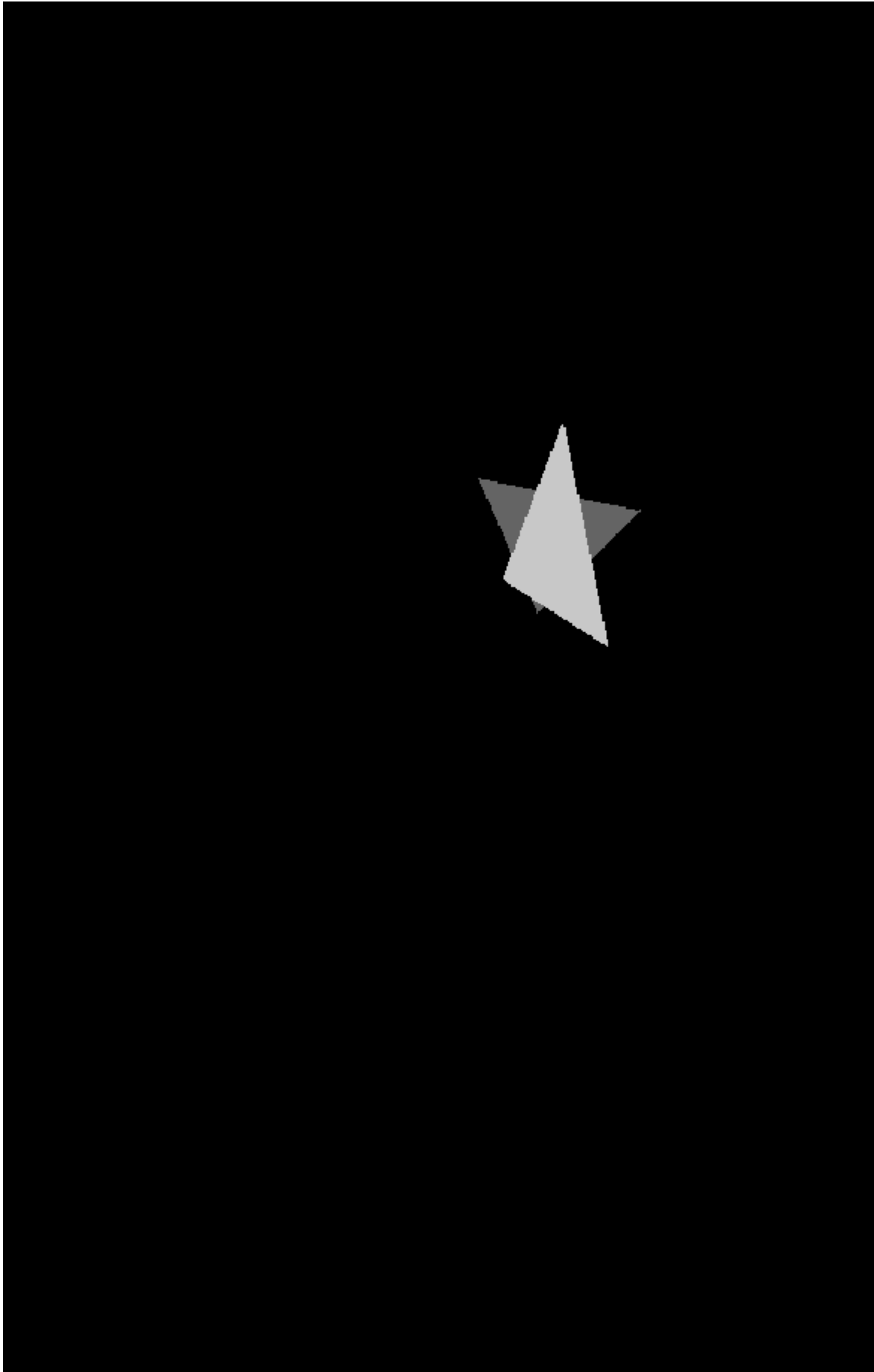
2.0.6 Funkcja do wyznaczania odległości punktu $[x,y,z]$ od płaszczyzny o równaniu $Ax+By+Cz+D=0$

% odleglosc mierzona jest wzdluz prostej o wektorze kierunkowym [l,m,n]
% zwraca rowniez punkt przeciecia z plaszczyzna

```
function [d,x1,y1,z1] = dist2plane(A, B, C, D, x, y, z, l, m, n)

    ro = (A*x+B*y+C*z+D)/(A*l+B*m+C*n);
    x1 = x-l*ro; y1 = y-m*ro; z1 = z-n*ro;
    d = sqrt((x-x1)^2+(y-y1)^2+(z-z1)^2);
end
```





2.0.7 Funkcja pomocnicza z laboratorium 04 wypełniająca kontur

```
function Im2 = floodfill1(Im, y, x, T, C)
    [H, W] = size(Im);

    % sprawdzenie zakresu
    if y < 1 || y > H || x < 1 || x > W
        error('Punkt startowy (y,x) jest poza obrazem.');
```

end

```
    % sprawdzenie jasności piksela startowego
    if Im(y,x) ~= T
        error('Piksel startowy (%d,%d) ma jasność %d zamiast T=%d.', ...
            y, x, Im(y,x), T);
    end

    % jeśli nowa jasność taka sama jak stara, nic nie trzeba robić
    if T == C
        Im2 = Im;
        return;
    end

    Im2 = Im;

    % kolejka pikseli do odwiedzenia
    Q = zeros(H*W, 2);
    head = 1;
    tail = 1;

    Q(tail,:) = [y, x];
    Im2(y,x) = C;

    while head <= tail
        cy = Q(head,1);
        cx = Q(head,2);
        head = head + 1;

        % 4-sąsiedztwo: góra, dół, lewo, prawo

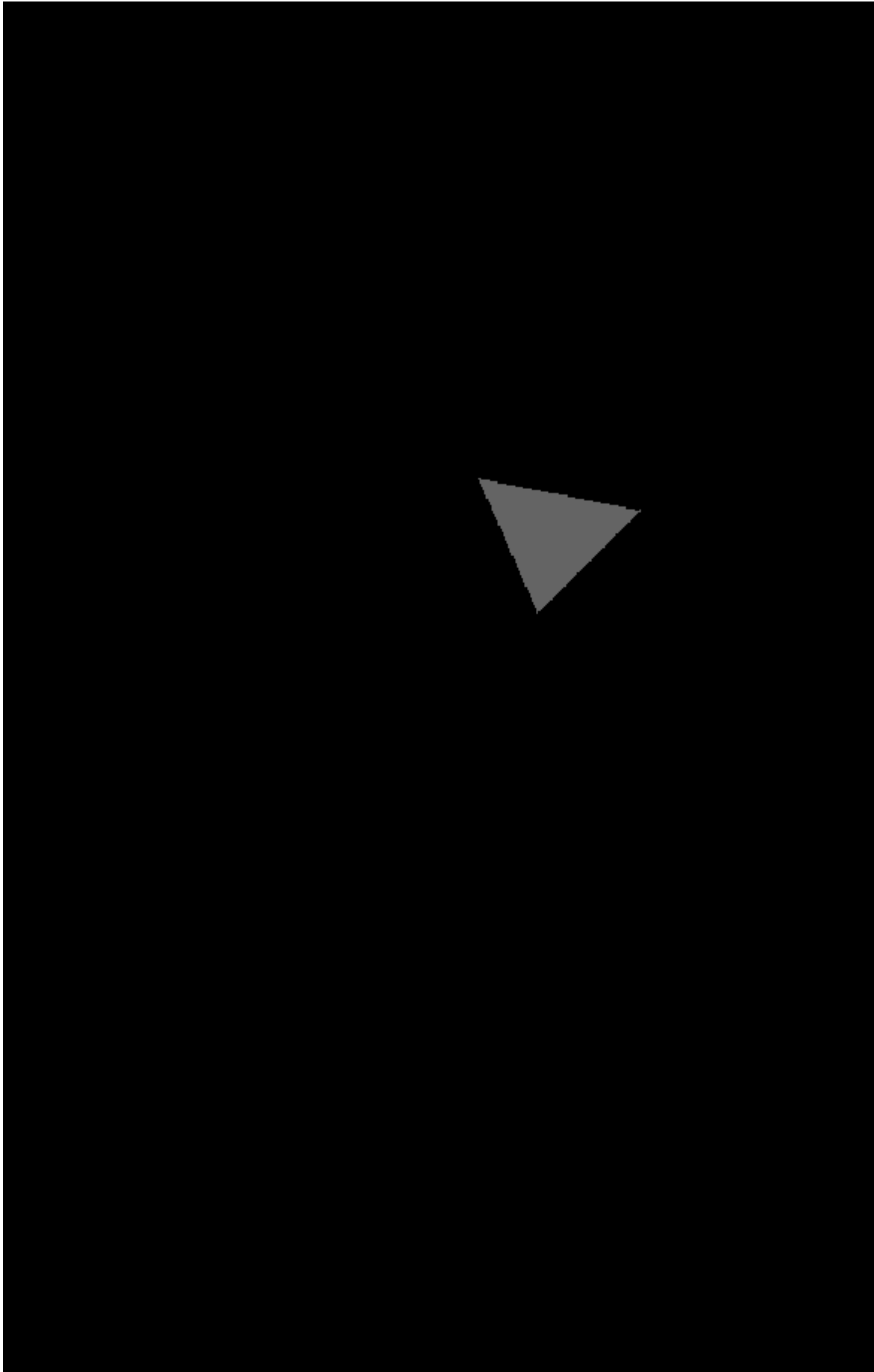
        % góra
        ny = cy - 1;
        nx = cx;
        if ny >= 1 && Im2(ny,nx) == T
            tail = tail + 1;
            Q(tail,:) = [ny, nx];
            Im2(ny,nx) = C;
        end

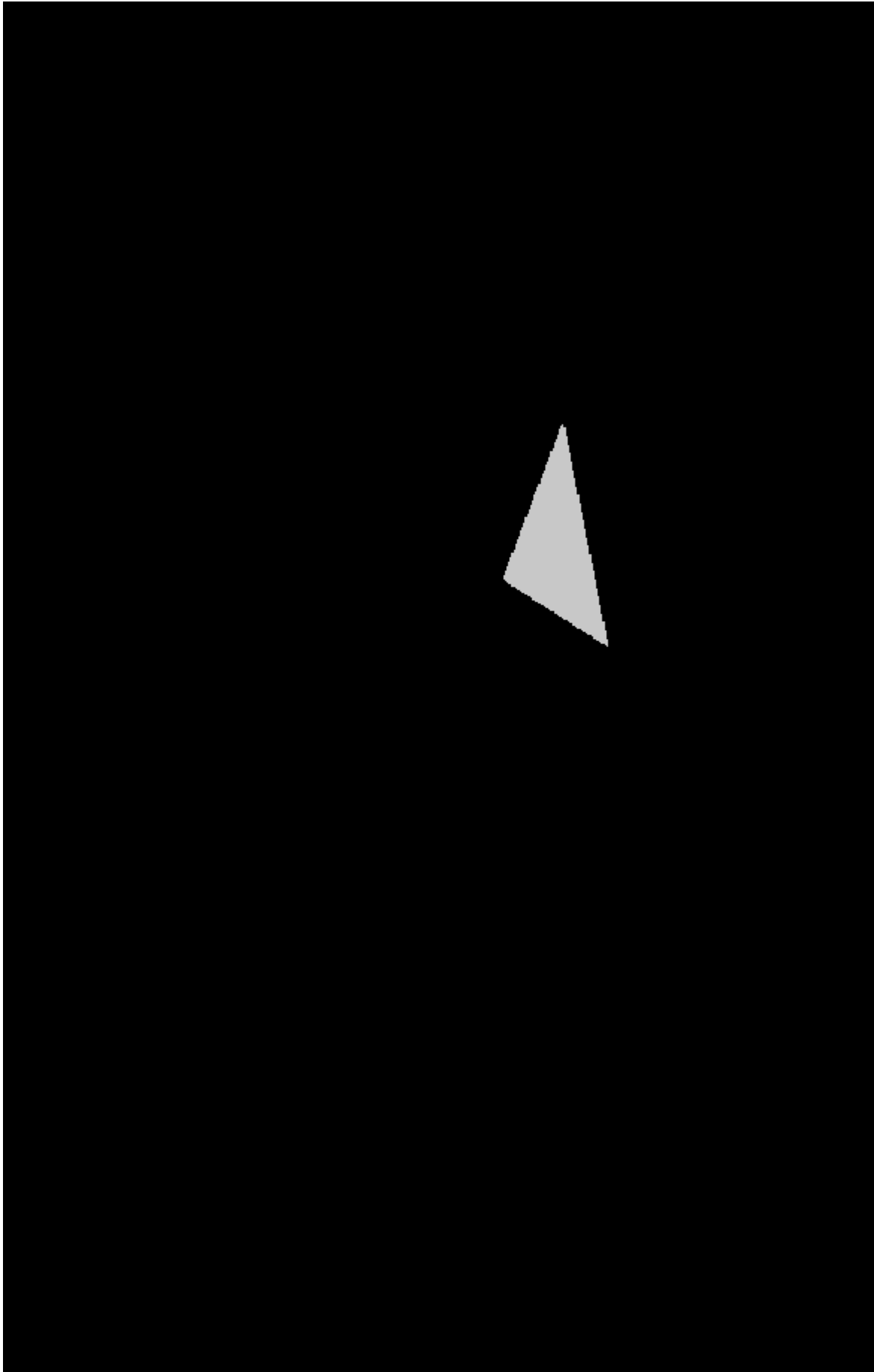
        % dół
        ny = cy + 1;
        nx = cx;
```

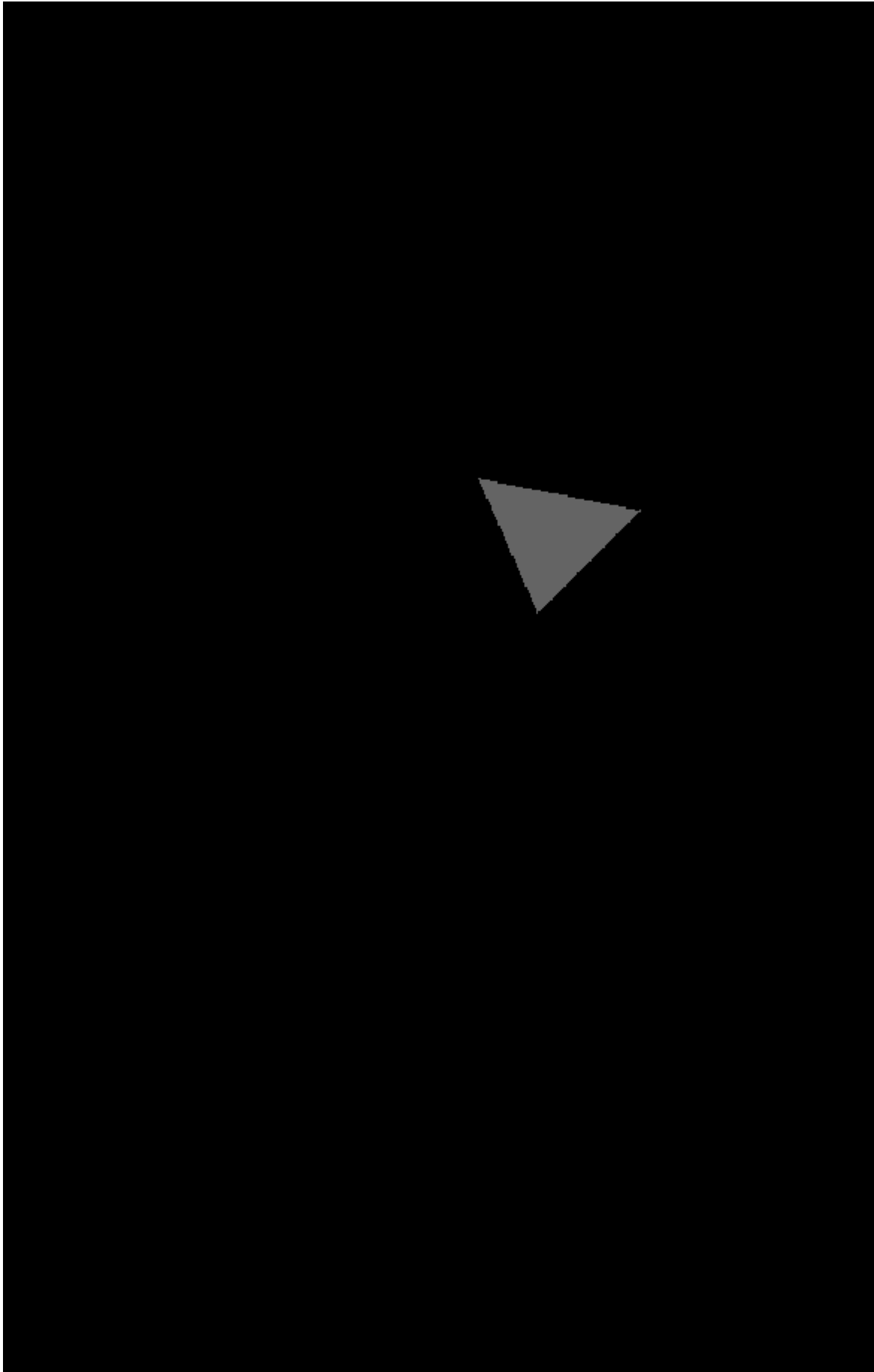
```
if ny <= H && Im2(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im2(ny,nx) = C;
end

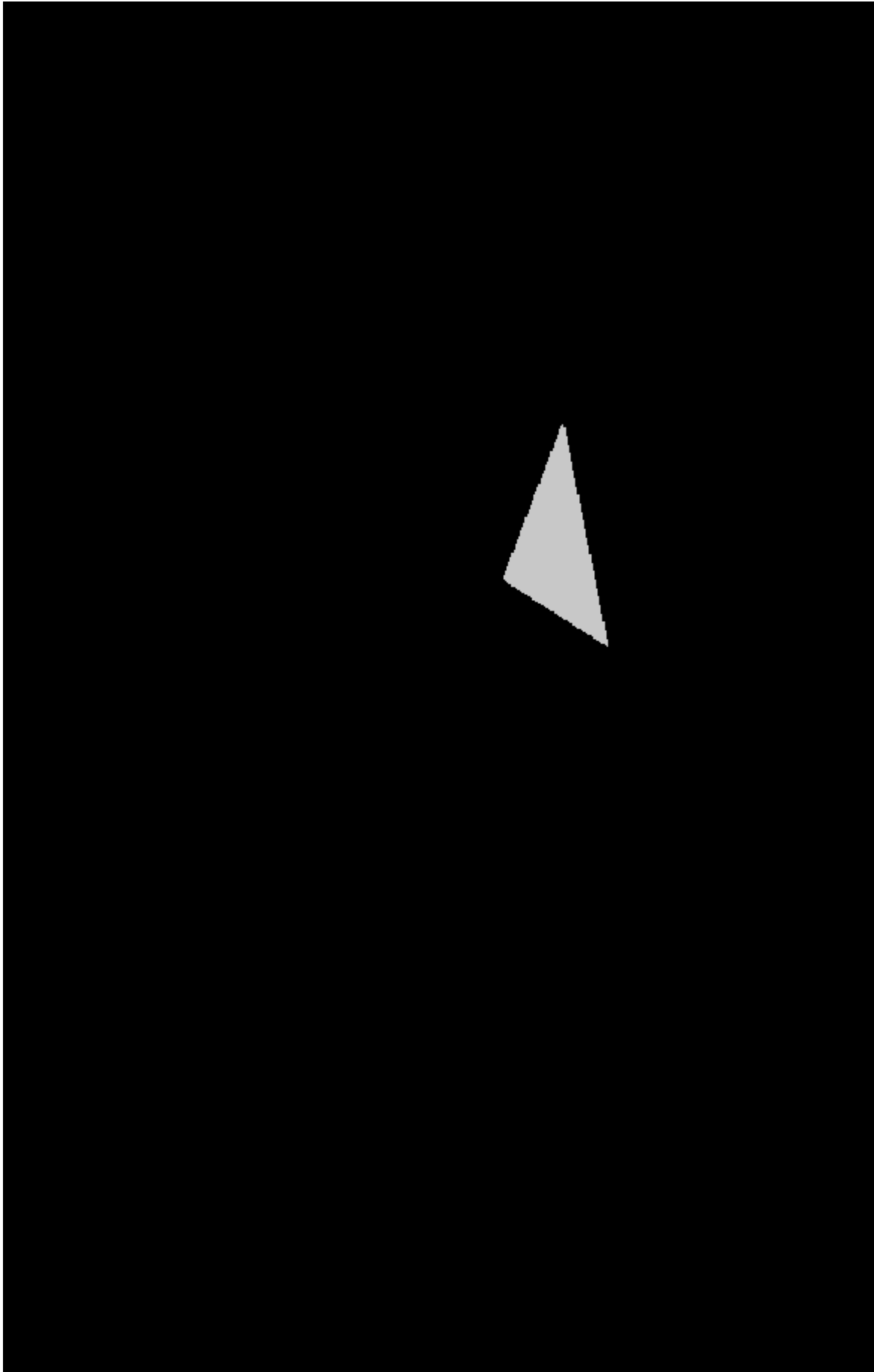
% lewo
ny = cy;
nx = cx - 1;
if nx >= 1 && Im2(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im2(ny,nx) = C;
end

% prawo
ny = cy;
nx = cx + 1;
if nx <= W && Im2(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im2(ny,nx) = C;
end
end
end
```







2.0.8 Funkcja pomocnicza z laboratorium 4 rysująca trójkąt i wypełniająca go

```
function [Imm1, Imm2] = wgLab4(p11,p12,p13,color)
    [x11,y11] = mat_to_pix(p11(1),p11(2),400,400,0.01,0.01);
    [x12,y12] = mat_to_pix(p12(1),p12(2),400,400,0.01,0.01);
    [x13,y13] = mat_to_pix(p13(1),p13(2),400,400,0.01,0.01);

    Imm1 = uint8(zeros(400,400));
    Imm1 = linia1(Imm1,y11,x11,y12,x12,color);
    Imm1 = linia1(Imm1,y12,x12,y13,x13,color);
    Imm1 = linia1(Imm1,y13,x13,y11,x11,color);

    xs1 = round((x11 + x12 + x13)/3);
    ys1 = round((y11 + y12 + y13)/3);
    Imm1 = floodfill1(Imm1, ys1, xs1, 0, color);

    Imm2 = Imm1;
end
```

2.1 ZADANIE 1

% Wygładzenie obrazu trzykrotnie operatorem uśredniającym 7x7.

```
I = imread('alpy.png');
[Y, X] = size(I);
I = double(I);

H = ones(7,7) / 49; % operator usredniajacy
I_edited = imfilter(I,H);
I_edited = imfilter(I_edited,H);
I_edited = imfilter(I_edited,H);
```

```
figure(1); imshow(I_edited/255);
```

% Podział obrazu na jednolite obszary za pomocą algorytmu *watershed*.

```
WSHed = watershed(I_edited);
WSBin = zeros(Y,X);
WSBin(:, :) = double(WSHed)/double(max(WSHed(:)));
```

```
figure(2);imshow(WSBin);
```

```
for j = 1:Y
    for i = 1:X
        if WShed(j,i)==0
            I(j,i)=0;
            I_edited(j,i)=0;
        end
    end
end
```

```
figure(3); imshow(I/255);
figure(4); imshow(I_edited/255);

% 2.2 ZADANIE 2

I = imread('alpy.png');
[Y, X] = size(I);
I = double(I);

H = ones(7,7) / 49; % operator usredniajacy

I_edited = imfilter(I,H);
I_edited = imfilter(I_edited,H);
I_edited = imfilter(I_edited,H);

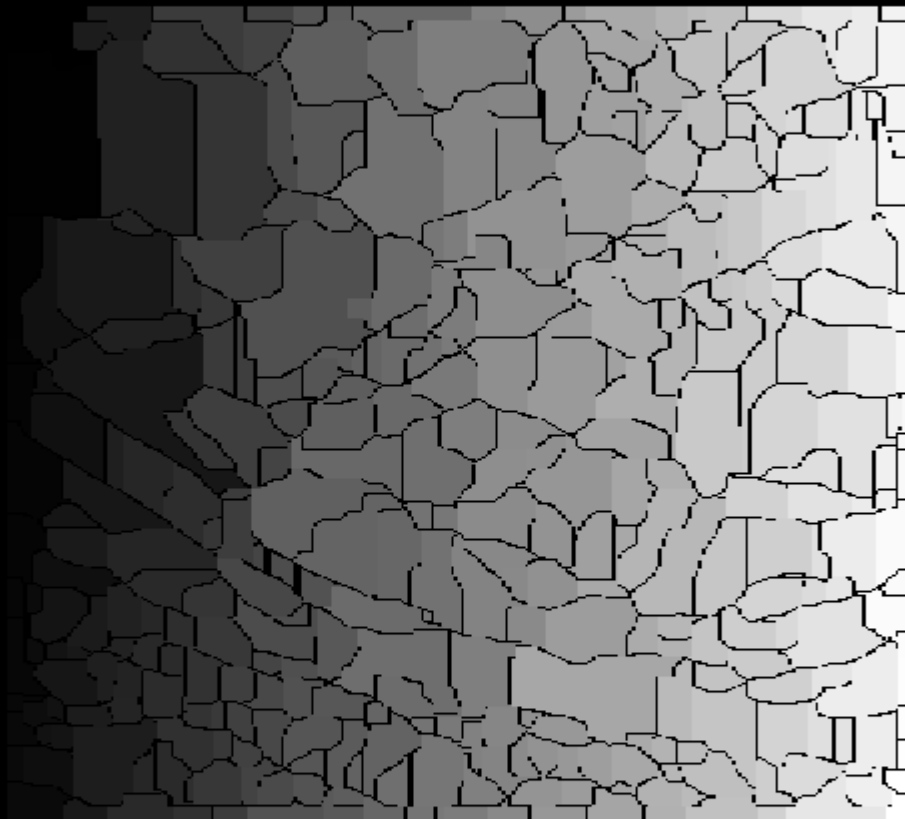
[CIX, CIY] = gradient(I_edited); % gradienty
CCI = abs(CIX) + abs(CIY); % laczny obraz gradientowy
CI1 = CCI/max(CCI(:)); % normalizacja do zakresu 0-1
figure(5); imshow(CI1);

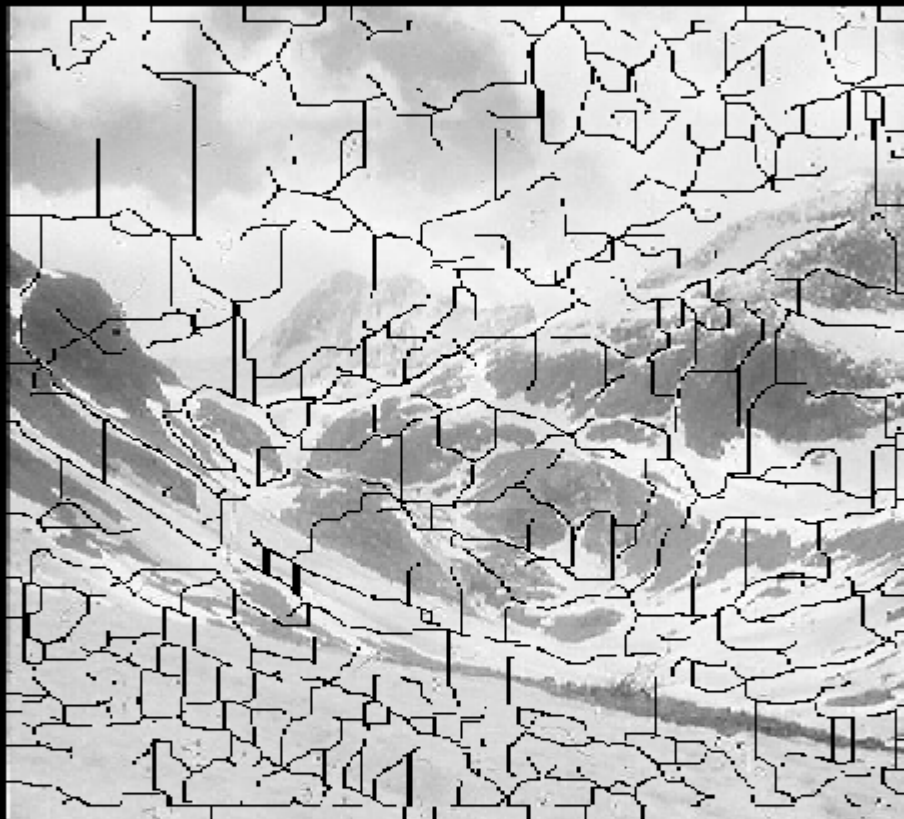
CI2 = imfilter(CI1,H);
CI3 = imfilter(CI2,H);
CI4 = imfilter(CI3,H);
figure(6); imshow(CI4);

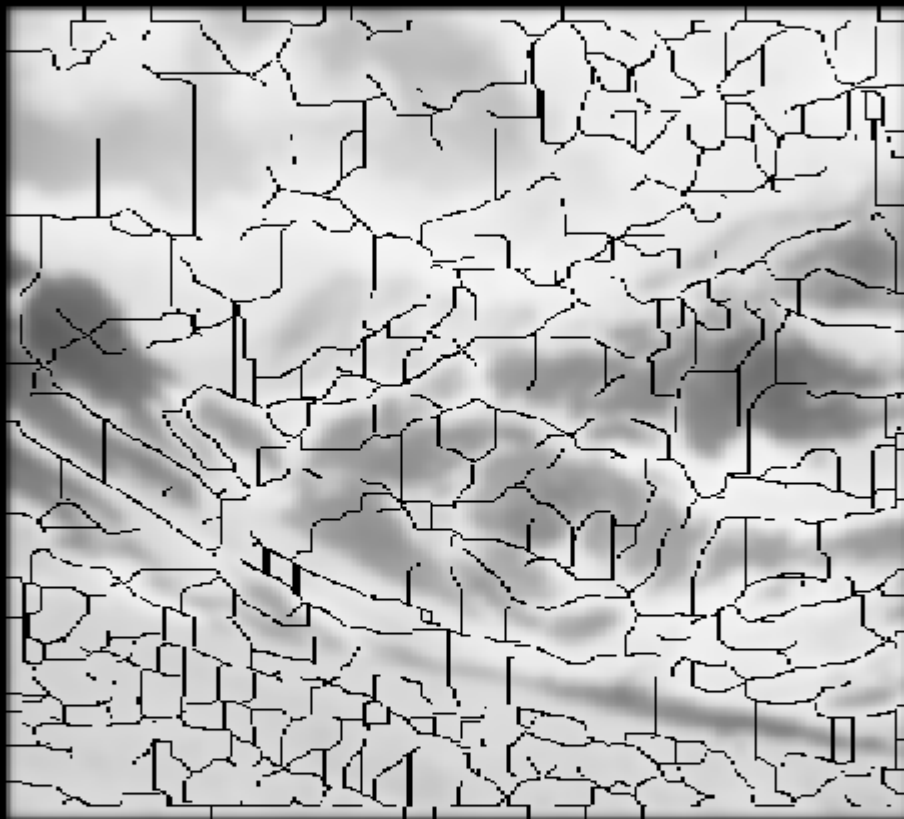
WSHed = watershed(CI4);
WSBin = zeros(Y,X);
WSBin(:, :) = double(WSHed)/double(max(WSHed(:)));
figure(7); imshow(WSBin);

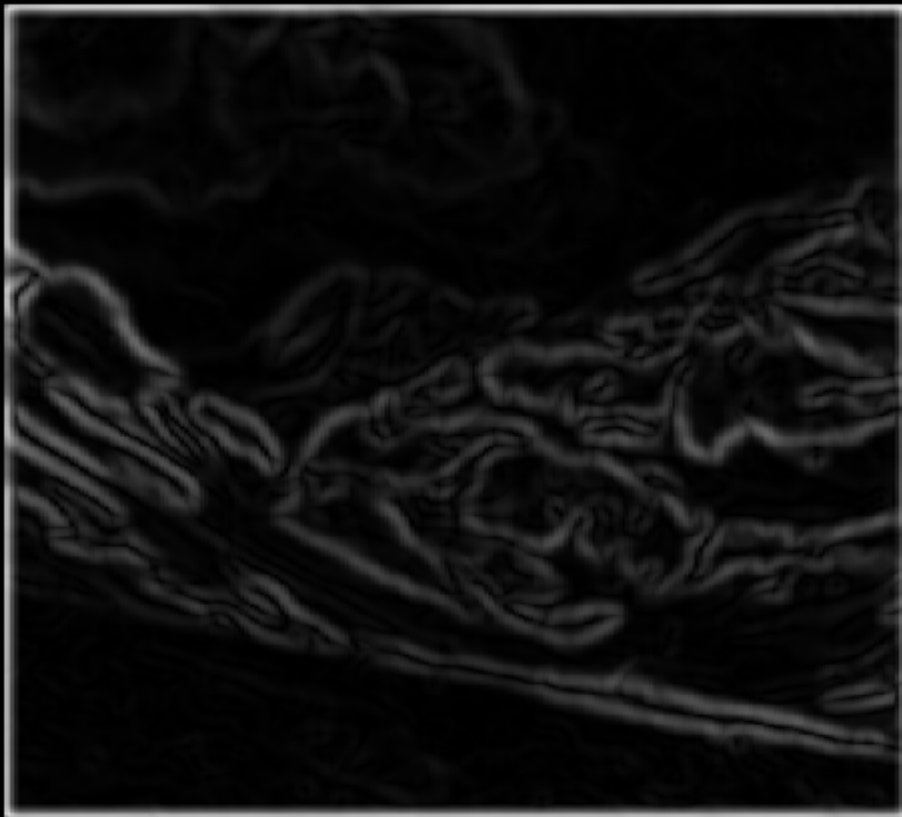
for j = 1:Y
    for i = 1:X
        if WShed(j,i)==0
            I(j,i)=0;
            I_edited(j,i)=0;
        end
    end
end
figure(8);imshow(I/255);
figure(9);imshow(I_edited/255);
```

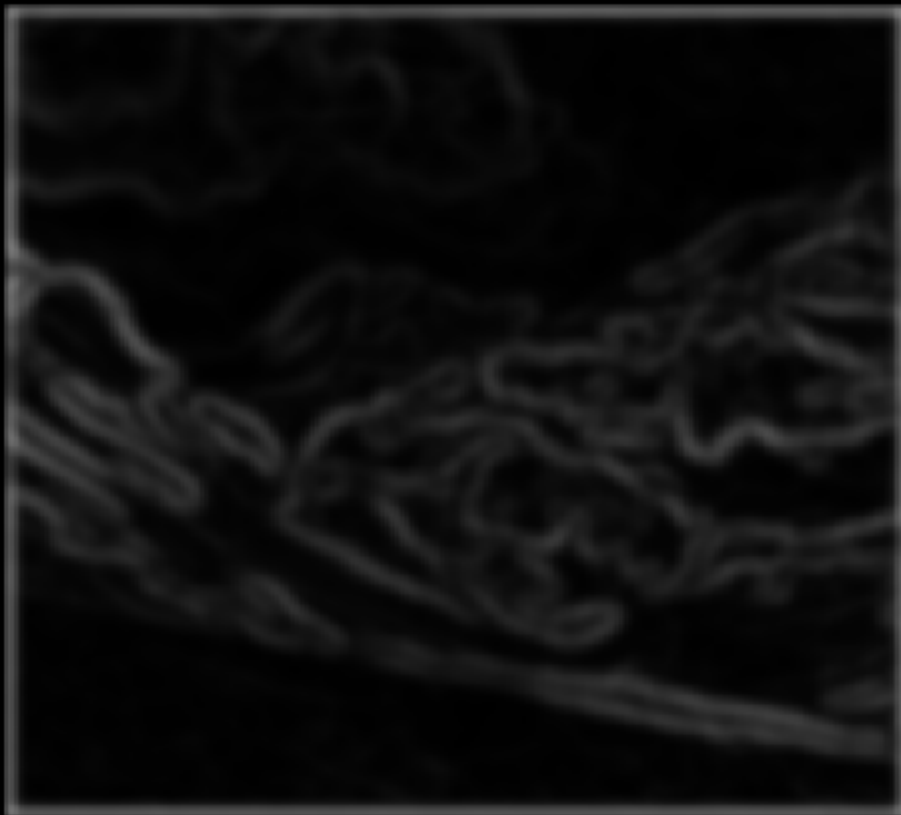


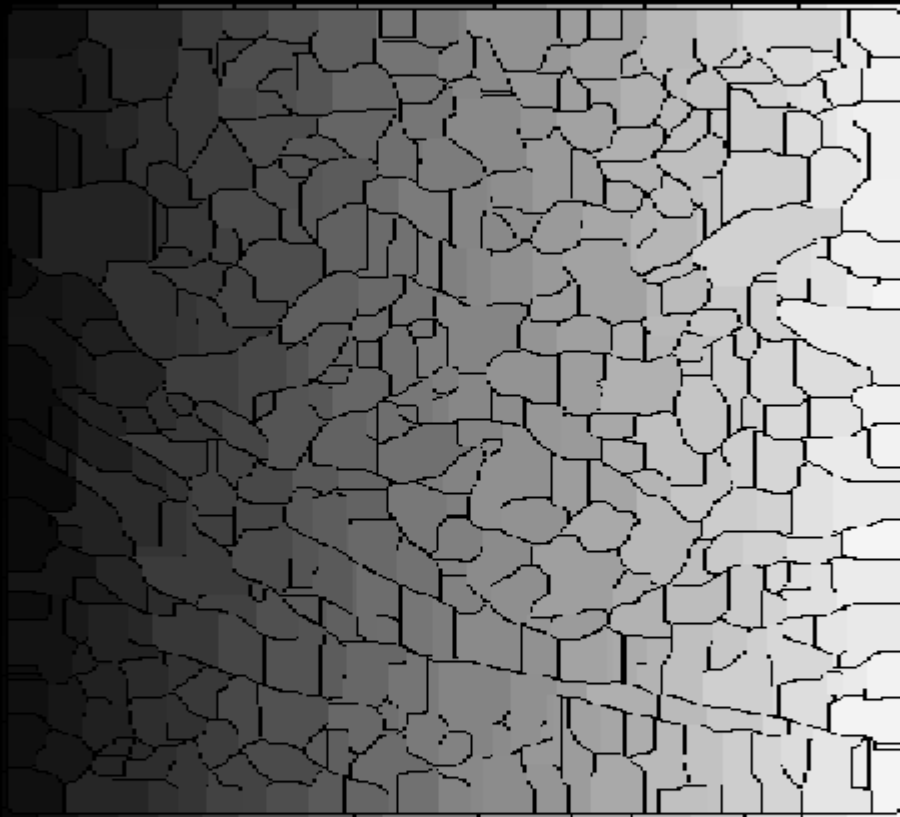


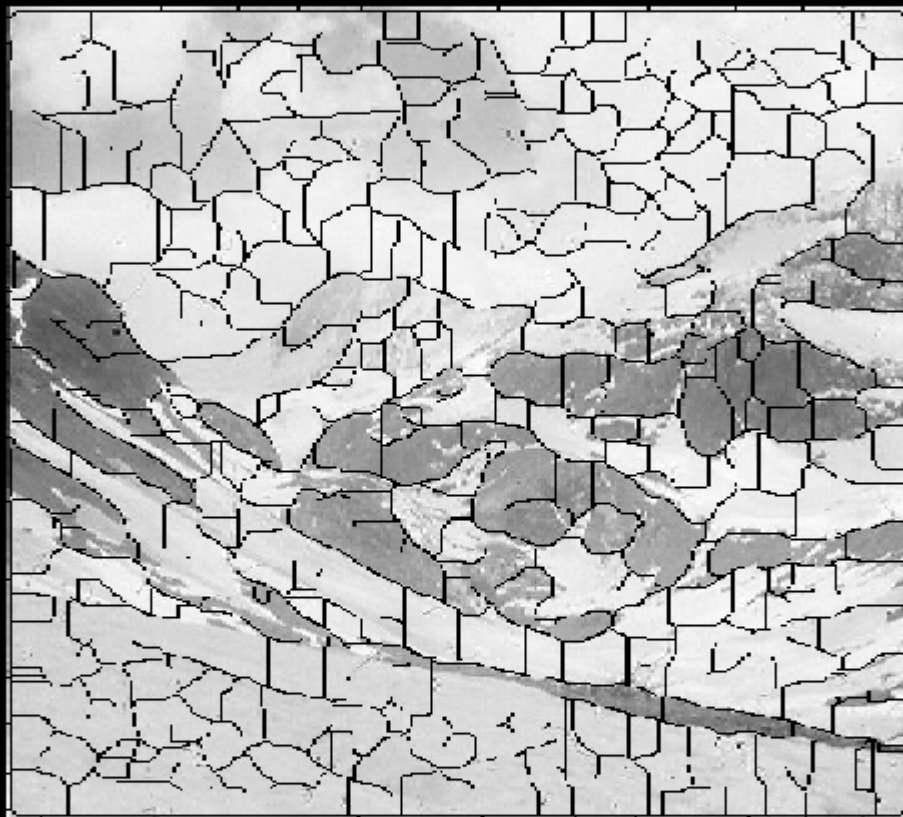


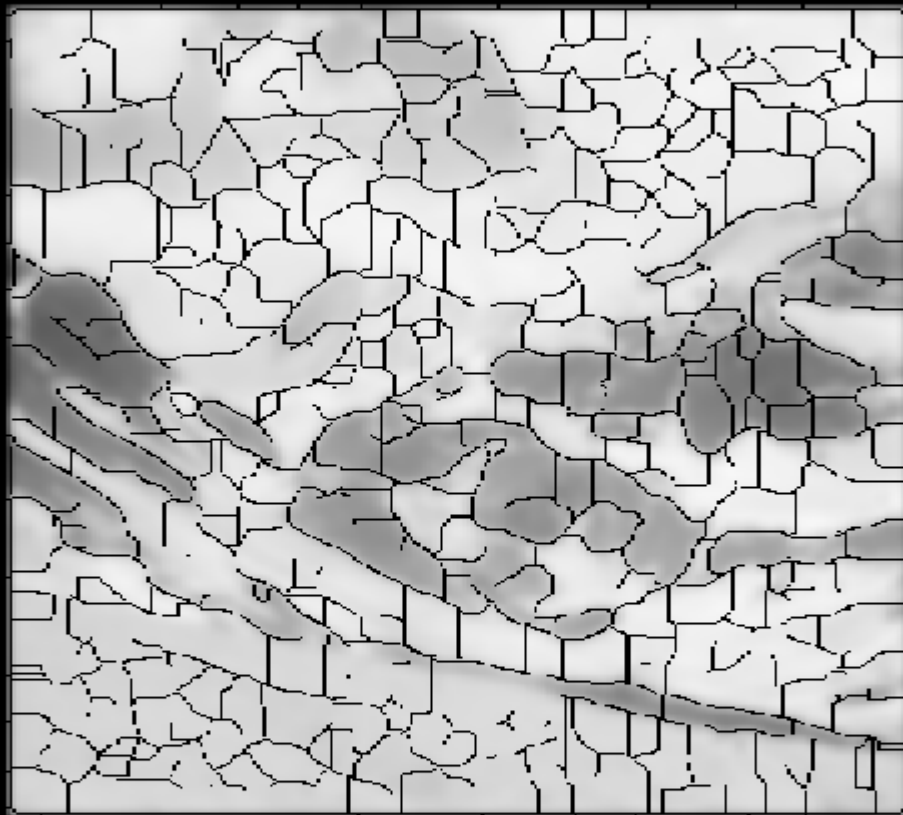












2.3 ZADANIE 3

% Rysowanie 2 trójkątów w tym samym miejscu, następnie generowanie łącznego
% widoku tych trójkątów.

% trojkat

p1 = [0;0;0;1]; p2 = [1;0;0;1]; p3 = [0;1;0;1];

% pierwsza transformacja

H1 = RPYT(30, 45, 10, 0.5, 0.3, 2.3);

p11 = H1*p1; p12 = H1*p2; p13 = H1*p3;

% druga transformacja

H2 = RPYT(-30, -45, -10, 0.3, 0.5, 2);

p21 = H2*p1; p22 = H2*p2; p23 = H2*p3;

HH = Persp(1);

[Imm1, Imm3] = wgLab4(p11,p12,p13,100);

[Imm2, Imm4] = wgLab4(p21,p22,p23,200);

figure(10); imshow(Imm1);

snapnow

close(gcf)

figure(11); imshow(Imm2);

snapnow

close(gcf)

figure(12); imshow(Imm3);

snapnow

close(gcf)

figure(13); imshow(Imm4);

snapnow

close(gcf)

% ***** z-buffer rzut rownolegly *****

FImR = uint8(zeros(400,400));

ZbuFR = 1000000*ones(400,400); % "nieskonczonosc"

[A1,B1,C1,D1] = plane(p11,p12,p13); % rownanie plaszczyny 1-go trojkata

[A2,B2,C2,D2] = plane(p21,p22,p23); % rownanie plaszczyny 2-go trojkata

for i = 1:400

for j = 1:400

[x,y] = pix_to_mat(j, i,400,400,0.01,0.01);

if Imm1(i,j)>0

[d,~,~,~] = dist2plane(A1,B1,C1,D1,x,y,0,0,0,1);

if d<ZbuFR(i,j)

ZbuFR(i,j) = d; FImR(i,j) = Imm1(i,j);

end

end

if Imm2(i,j)>0

[d,~,~,~] = dist2plane(A2,B2,C2,D2,x,y,0,0,0,1);


```
        if d<ZbuFR(i,j)
            ZbuFR(i,j) = d; FImR(i,j) = Imm2(i,j);
        end
    end
end

figure(14); imshow(FImR);
snapnow
close(gcf)

% ***** z-bufor rzut perspektywiczny*****

FImP = uint8(zeros(400,400));
ZbufP = 1000000*ones(400,400);
for i = 1:400
    for j = 1:400
        [x,y] = pix_to_mat(j, i, 400,400,0.01,0.01);
        if Imm3(i,j)>0
            [d,~,~,~] = dist2plane(A1,B1,C1,D1,x,y,0,-x,-y,1);
            if d<ZbufP(i,j)
                ZbufP(i,j) = d; FImP(i,j) = Imm3(i,j);
            end
        end
        if Imm4(i,j)>0
            [d,~,~,~] = dist2plane(A2,B2,C2,D2,x,y,0,-x,-y,1);
            if d<ZbufP(i,j)
                ZbufP(i,j) = d; FImP(i,j) = Imm4(i,j);
            end
        end
    end
end

figure(15); imshow(FImP);
snapnow
close(gcf)
```

Typowy koniec skryptu

```
% clear all; % usuniecie wszystkich zmiennych
% close all; % zamkniecie wszystkich wykresow
% clc % zresetowanie okna komend
```

Published with MATLAB® R2025b